

kibana

Kibana

Preliminary Steps - Elasticsearch Setup

Before being able to run Kibana you need to install both Elasticsearch and Kibana on the machine you're working on. The following steps will cover such aspects (in this case, a Windows installation will be considered).

First of all, download Elasticsearch from the following link (choose the link corresponding to your OS).

https://www.elastic.co/downloads/elasticsearch

Extract all the files in the desired folder.

Open a **cmd** and navigate to the Elasticsearch installation folder.

```
D:> cd D:/Elasticsearch-7.11.0/bin
```

Preliminary Steps - Elasticsearch Setup

Then, run **elasticsearch.bat.**

```
D:/Elasticsearch-7.11.0/bin> elasticsearch.bat
```

Open a web browser and digit **localhost:9200** in the search bar. If Elasticsearch is running correctly, a text describing the structure of your default cluster should appear.

```
{
   "name" : "LAPTOP-3S15C0JL",
   "cluster_name" : "elasticsearch",
   "cluster_uuid" : "WNcX7q8HRU2qa3n7Q5w7Lw",
   "version" : {...},
   "tagline" : "You Know, for Search"
}
```

Preliminary Steps - Kibana Setup

The same process should be carried out for Kibana.

Kibana can be downloaded at the following link.

https://www.elastic.co/downloads/kibana

Extract all the files in the desired folder.

Open **ANOTHER** **cmd** and navigate to the Kibana installation folder.

```
D:> cd D:/Kibana-7.11.0/bin
```

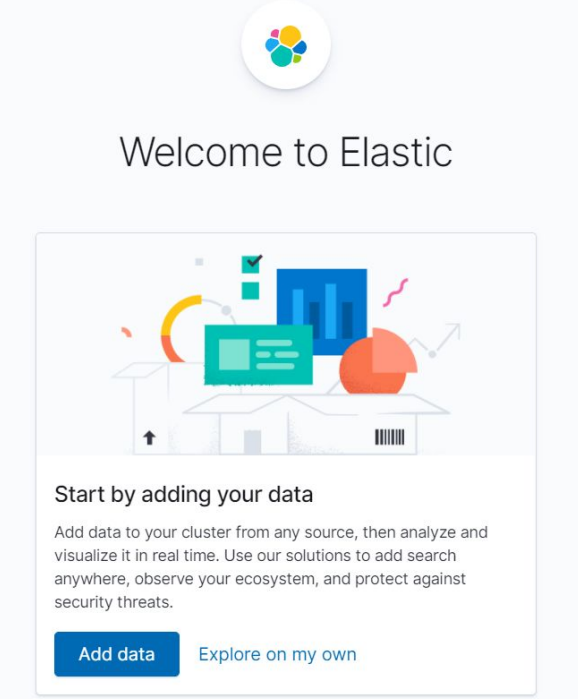
Preliminary Steps - Kibana Setup

Then, run **kibana.bat.**

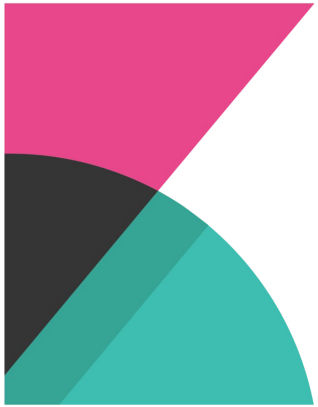
```
D:/Kibana-7.11.0/bin> kibana.bat
```

Open a web browser and digit **localhost:5601** in the search bar. If Kibana is running correctly, either a page asking for your data or the standard Kibana interface should appear.

Be careful! Leave both **cmds** open, otherwise the whole system won't work correctly.



What is Kibana?



Kibana is an open source data visualization dashboard for Elasticsearch.

It provides visualization capabilities on top of the content indexed on an Elasticsearch cluster.

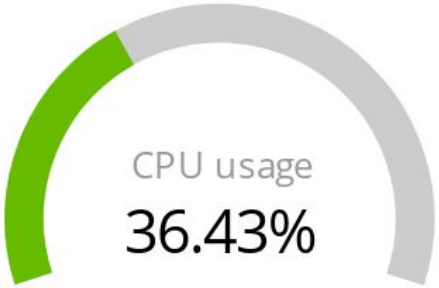
Kibana is simple and pretty intuitive to begin with. Despite such simplicity, it is highly customizable, allowing complex and detailed representations.

Kibana has many different kinds of subscriptions, although the main differences are between the free one and the others. The details about the differences are available on the **Kibana** official website.

What is Kibana?

Through **Kibana** you can aggregate, organize, filter and represent data using many different data types and representations.

A wide variety of graphs is available, although some data types, analyses and graphs are only available through a paid subscription.





What is Kibana?

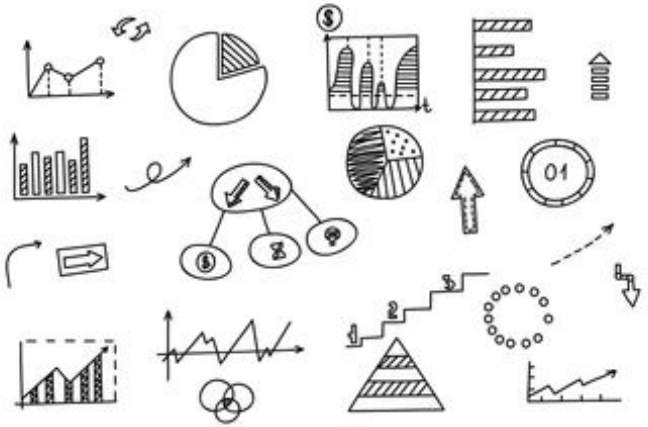
Kibana is really powerful. It allows the creation of customized dashboards combining together basic representations, location-based analyses, time series, machine learning and graphs/networks.



About Today

What we will take a look at today?

- Data Ingestion
  - Add Data
  - Agents & Fleets
  - Data Upload
  - Geospatial Data
- Data Overview
- Dashboard Creation
  - Interface Overview
  - Lens
  - Maps
  - Time Series Visual Builder (TSVB)
  - Filtering

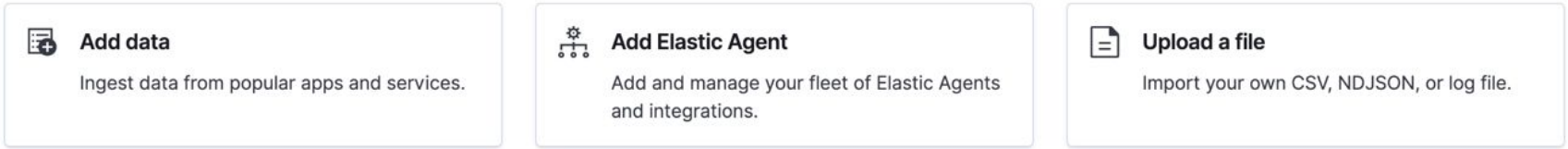


Data Ingestion - Overview

As mentioned before, the first step building dashboards or representations through Kibana, is to upload your data.

Kibana has many different kinds of upload to choose from, namely

- **Add Data** - Ingest data from popular apps and services.
- **Add Elastic Agent** - A centralized way to manage data integration (this feature is currently in beta).
- **Upload a File** - Import a CSV, JSON or log file (this feature is currently “experimental”).

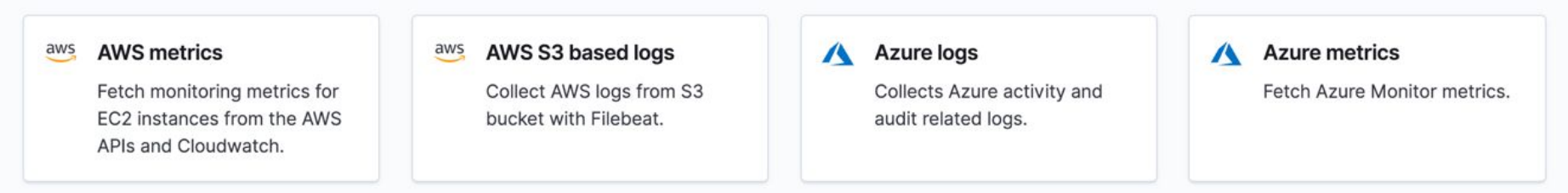


Data Ingestion - Add Data

Following, an overview of the upload processes listed before is provided.

Add Data

Import data from other services through some modules to periodically collect the data and send it to Elasticsearch. The picture below provides some examples of such modules.



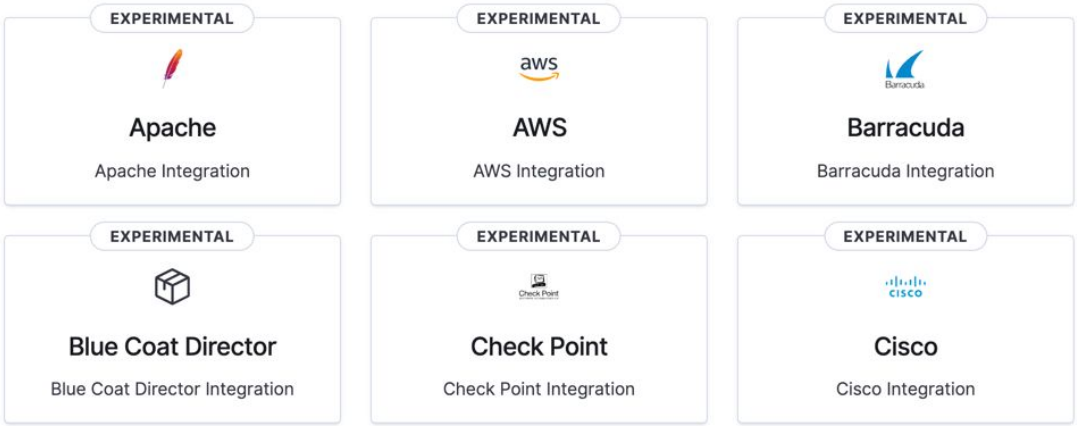
Data Ingestion - Elastic Agents

Add Elastic Agent

It offers a centralized way to set up your integrations through integrations modules. The integrations ship with dashboards and visualizations, so you can quickly get insights into your data.

Through **Fleet** mode, you can add and manage integrations for popular services and platforms, providing an easy way to collect your data.

The picture on the right exemplifies some of the available modules.



Data Ingestion - Fleet

The previous slide mentioned **Fleets**. What are they?

More in general, whenever you want to manage integrations you can either do it in **Standalone** or **Fleet** mode.

Standalone mode requires you to manually configure and manage the agent locally.

On the other side, **Fleet** mode offers several advantages, namely

- A central place to configure and monitor your Elastic Agents.
- An overview of the data ingest in your Elasticsearch cluster.
- Multiple integrations to collect and transform data.

A detailed user guide to **Fleet** mode is available here.

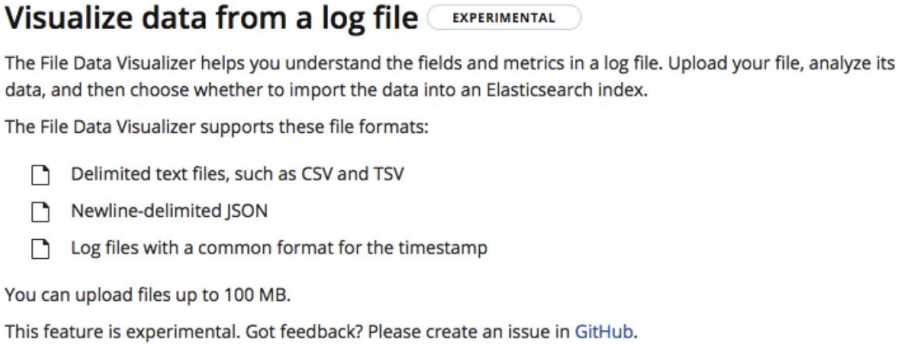
Data Ingestion - Upload a File

Upload a File

Kibana supports three main types of data, namely CSV, JSON and log files.

The file can be up to 100MB, although the system can be configured to upload files up to 1GB.





Data Ingestion - Geospatial Data

Geospatial data are treated in a different manner.

They are indexed as **geo_point** (a field accepting latitude-longitude pairs) or **geo_shape** (a data type that facilitates the indexing of and searching with arbitrary geo shape).

Depending on the format you have to pick a different tool.

```
{
  "text": "geo_point",
  "location": {
    "lat": 41.12,
    "lon": -71.34
  }
}
```

```
{
  "text": "geo_point",
  "location": "41.12,-71.34"
}
```

```
"geo_shape": {
  "location": {
    "shape": {
      "type": "envelope",
      "coordinates": [ [x,y], …]
    },
    "relation": "within"
  }
}
```

Practice - Upload your First Dataset

In this first practice tutorial, we will add the data to Kibana using the last option provided. The data will be about NYC Restaurant Inspection. The following examples will be built on top of this Dataset.

Before proceeding to upload your first dataset, we have to perform a small trick to upload files greater than 100MB. Therefore

- From the default screen displayed by Kibana, press “**Explore my Own**”.
- Click on “**Manage**” in the top-right of the screen.
- On the left side of the screen, under the voice “**Kibana**”, click on “**Advanced Settings**”.
- A page containing a lot of setup parameters will be displayed. Do not worry, you won’t have to look for the one we need by hand. Press CTRL+F (or CMD+Space) to display the search bar in the browser. In the search bar, write “**File Data Visualizer maximum file upload size**”.
- The default value associated with such parameter is 100MB. Change it to become **512MB**.
- Press “**Save Changes**” at the bottom of the screen.
- Click on “**Elastic**” in the top-left corner of the screen to go back to the data ingestion page.

Practice - Upload your First Dataset

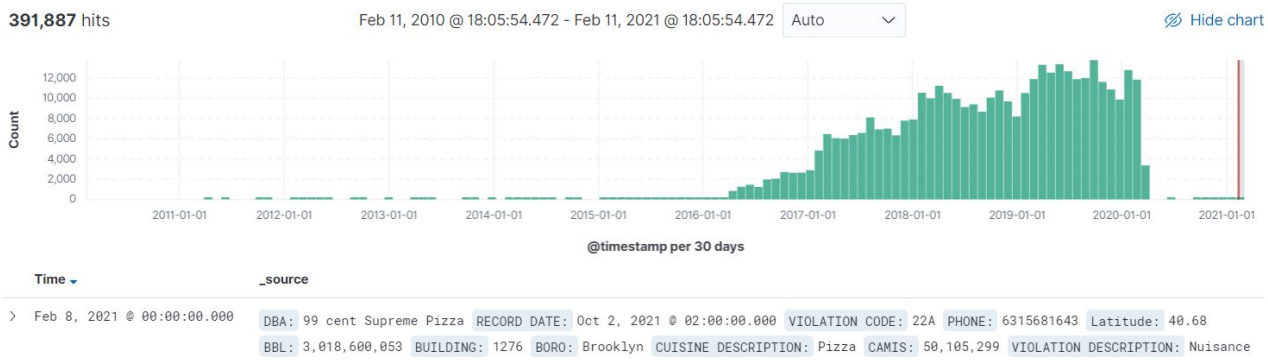
After you are done with the setup, you can finally upload your first dataset, therefore

- Download the dataset here (complete dataset here).
- Among the options displayed, under “**Ingest your data**”, click “**Upload a File**”.
- Drag and drop the excel file in the lowest part of the screen.
- A preliminary analysis of the dataset (by default, the analysis of the top 1000 tuples) is provided.
- Press “**Import**” at the bottom of the screen.
- You will be asked to provide an index name. Use “**nyc-restaurants**”. Be careful that capital letters are not allowed, so if an error is displayed, that’s the problem (probably).
- Press “**Import**” again.
- It will take a few moments to upload the data and create the index, be patient.
- A recap screen should have appeared, telling you the upload has been completed.
- Congratulations, you have successfully uploaded your first dataset.

Practice - Overview of your First Dataset

Now, let's check that our data has been correctly uploaded in Kibana.

- Press on the ☰ **Menu** icon at the top-right of the screen and look for "**Analytics**".
- Click on "**Discover**".
- There will be two different forms, a "**Search**" one and a "**Date**" one.
- Click on the "**Date**" one and under the "**Quick Select**" voice set the forms to select the last 10 years of data.
- Click on "**Apply**", then a graph providing the count of the uploaded data should appear.



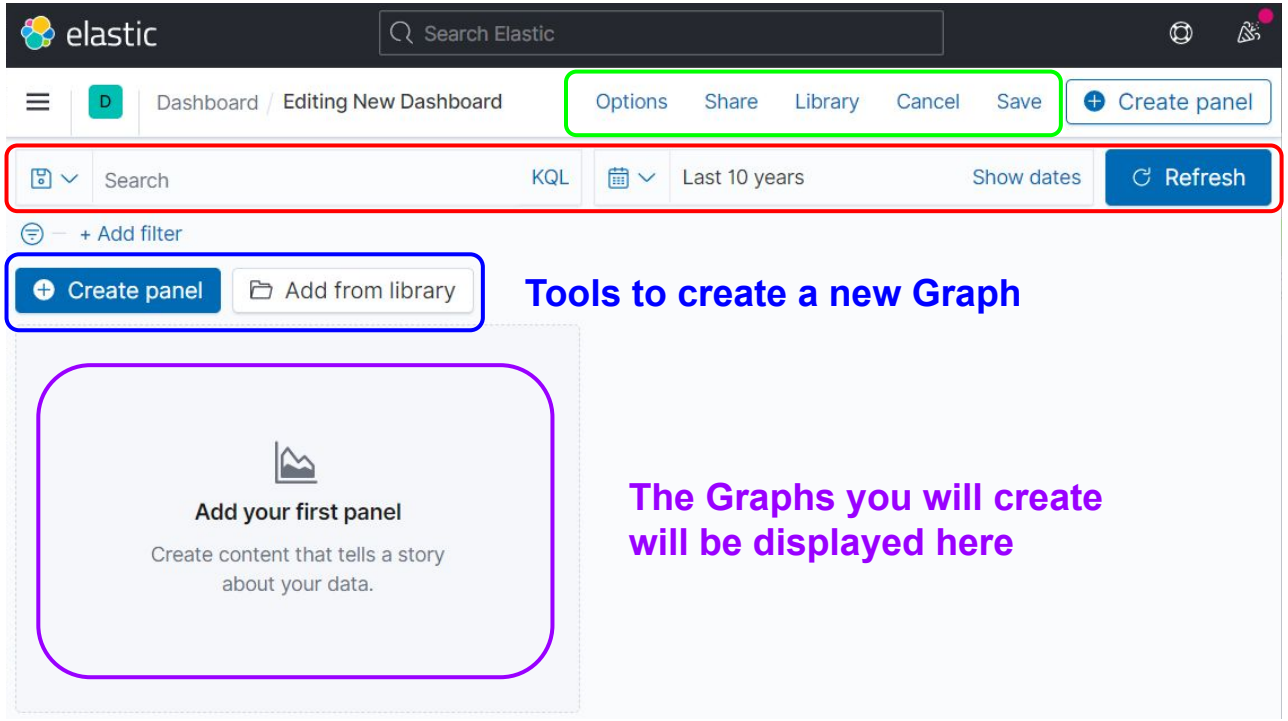
Practice - Create your First Dashboard

Now that we have our dataset uploaded, it's time to work on the representations. Therefore, we will now proceed to create our first Dashboard.

- Press on the ☰ **Menu** icon and look for "**Analytics**".
- Click on "**Dashboard**"
- Click on "**Create new Dashboard**"

The interface should now cite "Editing new Dashboard" at the top of the screen. We will analyze the elements provided by the interface in the following slides.

Practice - Create your First Dashboard



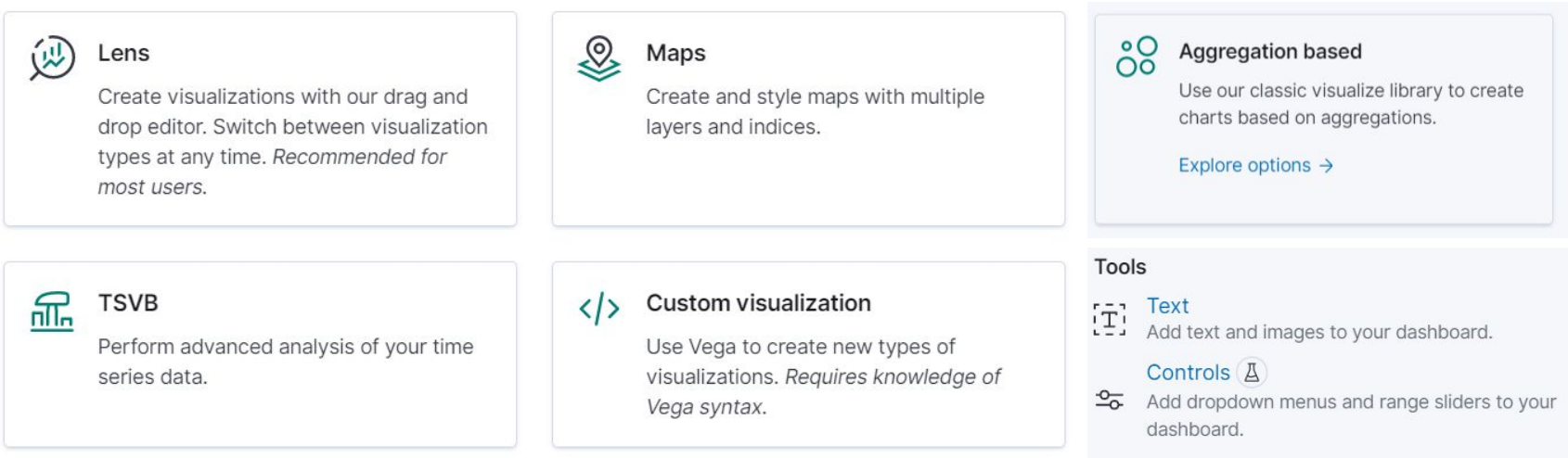
Dashboard Management

Time and Fields Filters

Practice - Create your First Dashboard

Press on “**Create Panel**” in the top-right (or top-left) of the screen.

A box with many different options will be displayed, as shown in the picture below.



Practice - Create your First Dashboard

For now let's focus on the simple things and ignore the most complex one. We will take a look at them later. Just click on "**Lens**".

"**Lens**" is the simplest and most intuitive representation tool provided by Kibana.

- At the top, you can see the "**Time Filters**" and "**Fields Filters**".
- On the left, you can see the list of fields we can use to display our data. The list also distinguishes between "**Fields**", "**Meta-fields**" and "**Empty Fields**".
- On the right, you can see some fields that represents the data you've chosen for the current representation.
- In the middle, you have some setup parameters and the preview of your representation. We will see that if we add some fields to the graph, the "**Lens**" tool will automatically provide advices on the kind of representation to use.

Practice - Create your First Dashboard

To become familiar with the “**Lens**” tool, we will now build some simple graphs. You will be free to explore this tool during the exercise session.

Let’s begin to create a panel containing the amount of tuples we have uploaded in Elasticsearch.

- Before proceeding, be sure to have 10 years in the “**Time Filter**”.
- Drag and drop the **Records** field in the central box.
- A small set of suggested graphs will appear on the bottom.
- By overlaying them with the cursor, you will be able to see their name.
- Press the “**count overall**” one.
- Now, press “**Save**” on the top-right of the screen.
- A window asking for a name, a description and some tags will be displayed. Name the panel “**Count Overall**”.
- Press “**Save and Return**”
- Your graph will be now added to your dashboard

Practice - Create your First Dashboard

On the top of the screen, you can notice how our dashboard hasn't been saved at all. Indeed, the name should be something like "**Editing New Dashboard (unsaved)**".

Let's save our dashboard so that we won't lose our work. Click on "**Save**" in the top-right of the screen. A window similar to the previous one will be displayed. Just name the Dashboard "**My First Dashboard**" and click "**Save**".

Now, if you click the "**Dashboard**" option, under "**Analytics**", in the **Menu**, you will be able to see that our dashboard is displayed in the list. From there you open the dashboard by clicking its name. Then you will be able to edit the dashboard by clicking on "**Edit**" in the top-right of the screen. Don't forget to save your changes every time you update or add new panels.

Practice - Create a Panel

Let’s now add a few additional graphs to our dashboard

- Press “**Edit**” on our dashboard. Then, “**Create Panel**” and “**Lens**”.
- Drag and drop the “**SCORE**” field in the middle of the screen.
- On the right, the **@timestamp** field will be automatically set as the “**Horizontal axis**”. Remove it by pressing the red cross symbol on its right. The graph will automatically change displaying a random graph. Don’t you worry, we will fix it right away.
- From the fields list, drag and drop the “**Council District**” field as the “**Horizontal axis**”.
- The graph will be then updated. You should have “**Average of Score**” on the y-axis and “**Council District**” on the x-axis.

Before proceeding to create our new panel, let’s explore a little bit the functionalities we have in this section of the dashboard creation.

Practice - Create a Panel

First of all, try to click on one of the fields available on the left side of the screen. A preview of the "**Distribution**" and "**Top Values**" of the 1% of the data will be displayed. That way you're provided with an overview of a small sample of the data.

On the right side of the screen, click on "**Average of Score**". A small window with a set of functions will be displayed. The blue ones are the ones suited for the field we are currently analyzing, while the ones greyed out are the ones not suited. Clicking one of the latter will automatically change the chosen variable with another one for which such function is suited. Try to explore some of the suited ones by clicking on them and see how the graph changes (e.g. try to click "**Maximum**" or "**Sum**").

Still in the same window, you can change the variable and configure some features of the chosen axis of the graph. Now, close the window for the "**Vertical Axis**". These are just some examples, each field has its own customizable axis depending on its type.

Practice - Create a Panel

On the right, click on “**Council District**”. As before, a window with a set of functions will appear and some of these are similar to the ones explained previously. The new one is the “**Granularity**” one, through which you can change the way the data is displayed on the x-axis. The greater the granularity, the more detailed the data on the x-axis will be; lowering such value will aggregate the values on the x-axis. Try playing with it a little bit to see how the graph changes. Then, you can close the window.

On the top of the graph, try to change the kind of representation of the graph (e.g. try to change the graph to an “**Area**” graph).

The last thing we will see here are the x-axis filters. In particular, try dragging over the graph, select an area and see how the representation changes. You can notice how, on the top-left of the screen, you will see “**Council District: X to Y**” (where X and Y are the lowest and highest value respectively). If you want to remove the filter, just click on the cross symbol on the newly added filter.

Hands On - Create a Panel

Now, let's save the graph as "**Score VS District Number Area Graph**" and take a look at our dashboard. It's a little bit empty, isn't it?

Now it's your time to enrich the dashboard. Use the following variables to create 2 or 3 graphs to add to your dashboard using the "**Lens**" creation tools.

- **BORO** → Borough in which the entity (restaurant) is located.
- **INSPECTION DATE** → Date of the inspection.
- **SCORE** → Inspection Score.
- **CRITICAL FLAG** → Indicator of a critical violation.
- **INSPECTION TYPE** → A combination of the inspection program and the type of inspection performed.

e.g. try to create a graph displaying the average score per borough, or a graph displaying the frequency of each inspection type, etc.

Practice - Explore your Dashboard

After you are done with your panels, just move back to your dashboard.

Let’s try to change the time filter. Let’s set it between today and a year ago. The dashboard should update itself to include all corresponding data entries.

Let’s try to do the same thing we did while creating a panel. Select an area on one of the graphs. Notice how the filter has been added in the filter area on the top-left side of the screen and all the dashboards have been updated consequently.

What if you click on one of the borough in the graph displaying the average score per borough?

We have just learned how to create simple and intuitive dashboards through the “**Lens**” tool. Following, an overview of the other tools will be provided.

Practice - Maps

Now, let's add a new kind of panel to our dashboard.

- Press on "**Create Panel**" in the top-right (or top-left) of the screen.
- This time, select "**Maps**". A big map of the world will be displayed.
- On the right, press on "**Add layer**".
- On the right, a menu with multiple options will appear. From there, we can directly upload data (although a maximum size of 50MB is set) or use the one we already uploaded.
- Since we already have our dataset upload, press on "**Documents**" (its description also states how this option imports data from elasticsearch).
- As the "**index pattern**", select our dataset, named "**nyc-restaurants**". The tool will automatically detect the fields suited for the current representation. As you may notice, most of the tuples in the dataset have a correct value in the "**location**" field, while a small amount of them is incorrect, and therefore positioned in the middle of the ocean.

Practice - Maps

- Try to zoom in the area around New York city, in which around 385’000 restaurants are located. Notice that, as you zoom in, the different restaurants will be divided depending on their location in the different districts of New York and surroundings.
- On the left, you’ll still be able to see some visualization options.
- For now, let’s click on “**Add layer**” at the bottom-right of the screen.
- A new set of customization options will become available.
- Let’s name this new layer “**nyc-restaurants-clustered**”
- The most important field to mention is the “**Visibility**” one, through which you can set the visibility of the clusters depending on the level of zoom on the map. Try and play out with such field.
- Following, in the “**Tooltip Fields**” it is possible to add some fields that can be visualized on the map when inspecting the individual elements. Try to add the “**SCORE**” field and zoom on the map until you can see each restaurant individually. By overlaying with the cursor, their score value should appear.

Practice - Maps

- You can also add some filters to the data through KQL (Kibana Query Language) and customize the representation of the current layer. For example, if you select “**icon**” all the restaurants will be represented through a pin icon rather than a circle. Under the “**icon**” attribute, you can select “**By value**” rather than the default “**Fixed**” and then pick the “**SCORE**” field. The map will change the icons of the restaurants depending on their scores. By doing so, we are also able to visualize two additional attributes on the map named “**Top score**” and “**Top score %**”. Try to overlay over one of the aggregated symbols on the map to visualize them.
- For now, leave the layer as of now and press “**Save & close**” at the bottom-right of the screen.
- Then, press “**Save**” on the top-right of the screen. Name the map “**nyc-restaurants-map**” and save it. The map will then appear in your dashboard. From there you can still explore the map as before and also through the small buttons available in the map interface.

Practice - TSVB

Now, let's explore the "**TSVB**" option. This option allows the creations of more detailed time series analysis. We will mainly cover two of them, the "**Table**" and the "**Gauge**" one.

The "Table" representation is really useful to organize and visualize aggregated data.

- Press on "**Create Panel**" in the top-right (or top-left) of the screen.
- Pick the "**TSVB**" option. An interface with many different options will be displayed.
- Press on the "**Table**" one. Notice that no data is currently visualized.
- At the bottom of the table, select "**INSPECTION DATE**" as "**group by field**" and set the "**column label**" as "**Date**". A bunch of dates will appear in the table.
- Below the first box, in the second one, set the "**Label**" as "**Inspections per Day**" and leave "**count**" as our "**Aggregation**".
- Now let's set up our data to be visualized in the table. Click on "**Panel options**".
- In "**Data timerange mode**" select "**Entire time range**". Now the table will display some data.

Practice - TSVB

- To set the amount of rows we will show it's enough to change the value of the "**Rows**" field in the first bot at the bottom of the table preview. Let's put it to 100 for now.
- To sort the table depending on the values of the columns, it's enough to press on the name of the column. Try to sort the table in a descending order of "**Inspections per Day**".

Now it's your turn to explore the "**Table**" option.

- Add a new column to the table with the sum of all the scores given in a specific date and name it "**Sum of Given Scores**". To add a new column, you have to press on the ⊕ plus button, near the first column we created.
- Try to do the same with a column named "**Average Score per Day**".

- Then, save your table as "**Aggregation by Date**".

Practice - TSVB

As mentioned before, the next step will be the “**Gauge**” graph.

- Press on “**Create Panel**” in the top-right (or top-left) of the screen.
- Pick the “**TSVB**” option. An interface with many different options will be displayed.
- Press on the “**Gauge**” one. Notice that a gauge with a random number of values is displayed.
- As before, press “**Panel option**” and in “**Data timerange mode**” select “**Entire time range**”. Now the gauge will display the count of all the tuples we have imported from our dataset.
- Let’s display the number of scores greater than 30 assigned to the restaurants. Change the label of the gauge and name it “**Good Restaurants**”.
- In the “**Group by**” section, set the value to be “**Filter**”.
- In the “**Search**” filter just write “**SCORE**” and kibana will automatically suggest you what to write depending on the kind of variable you have. Then, complete the query by typing “**>= 30**”.
- Then, save the gauge as “**Good Restaurants Count**”.

Practice - TSVB

The previous exercise is using gauges at their best, since gauges are meant for a much wider set of applications, like informing the user whenever there are some critical conditions on the value of a variable, for example.

Let's assume we want to award the city of New York when it reaches 100'000 good restaurants.

- Edit the gauge we previously built.
- Click on "**Panel Options**".
- Under "**Style**" in "**Gauge Max**" insert 100'000.
- Under "**Color Rules**" add the following rules (Be careful, the order and the conditions matters!).
  - Color **Green** if metric is "**>= greater than or equal**" than 90'000.
  - Color **Yellow** if metric is "**<= greater than or equal**" than 89'999.
  - Color **Red** if metric is "**<= less**" than 25'000.
- Now the gauge should be **Yellow** since we have around 70'000 good restaurants
- Try to play with the "**Time Filter**" and set it to 1 year instead of 10 (be careful to restore it to 10 before proceeding with the rest of the tutorial)

Hands On - TSVB, Maps and Filters

Now it's your time to explore the "**Gauge**" type.

- Create two different gauges: one representing the number of inspections performed in the last year ("**Last Year's Inspections**"), and one representing the number of inspections performed in the last year, which achieved a score between 15 and 29 ("**Appreciated Restaurants**").

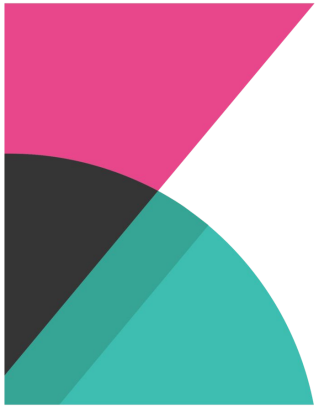
  Advice: KQL (Kibana Query Language) is quite intuitive, but also quite unique. To set a time value in the filter, use the following syntax (this is not the only option available, but it's the most intuitive one)

  now + **operator** (+ or -) + **quantity** (any number) + **time letter** (y = year, M = months, etc.)

  (e.g. now+1h (in an hour), now-10M (ten months ago), now-4w (four weeks ago), etc.)

Previously, we taught you how to deal with maps. What about improving them with what we learned about filters? Try to add some filters (like the ones provided for the gauge graphs) to the map we built.

Conclusions



Kibana is a very interesting tool to explore, organize and visualize huge amounts of data.

Kibana is easy to learn thanks to its intuitiveness and the tools it provides, but really hard to master. You should have notice it from the complexity it allows to manage while creating all the different panels.

But… Our journey isn't over yet. Now we will provide you with a user study and a dataset to test your skills.

Final Hands On - Motor Vehicle Crashes Case Study

As a final exercise we will have you create a dashboard about the motor vehicle crashes in New York.

Before proceeding, download the dataset here and upload it. Call the index "**vehicle-crashes-nyc**". After the dataset has been uploaded remember to check the fields through the "**Discover**" function.

In particular we want to create a dashboard with the following graphs.

- A map displaying the locations of the crashes in which at least 1 person died.
- Metrics displaying all the deaths in crashes per kind (cyclist, pedestrians, etc.) with alarms when the number of people killed reaches a high number (e.g. if more than 1500 people are killed).
- A time series representing the number of people killed per 4 weeks.
- A time series representing the number of people injured per 4 weeks.
- A Top N graph displaying the maximum for all the categories of injured and killed people.
- A Donut chart that represents the "CONTRIBUTING FACTOR" of the first vehicle.
- A Pie chart representing the percentages of the borough in which the crashes took place.

Final Hands On - Importing the Solution

If you didn't manage to create all the graphs don't you worry, now we will import the final solution.

- Download the file here (original dataset here).
- Then, at the top of the screen in Kibana you should have a search bar, type "**Saved Objects**" and press the voice "**Kibana / Saved Objects**".
- A list of all the objects created will be there.
- Press "**Import**".
- On the left, drag and drop the file you downloaded under "**Select a File to Import**".
- The system will automatically import the dashboard.

If you want to export your own dashboard, it's enough to

- Select the item(s) you want to export.
- Press "**Export**" and then "**Export**" again.

If you're exporting a dashboard, all the graphs it includes will be automatically exported.

Final Fix

Since the solution has been created with different names, you should add an alias to be able to display it. Therefore, go to the left menu, press on **Dev Tools**, insert and execute the following code.

```
POST /_aliases
{
  "actions" : [
    { "add" : { "index" : "name_of_your_vehicle_crash_cluster", "alias" : "veihicle-crashes-nyc" } }
  ]
}
```